COURSE CODE: CSA0308 | COURSE NAME: DATA STRUCTURES

CO1-AT2 TECHNICAL PRESENTATION

STUDENT NAME: Dhavanesh S S

REG NO: 1972260011

List of Questions & Academic Integrity

S. No.	Question Description
1	A student management system needs to store the marks of 1000 students efficiently. Apply the concepts of primitive and non-primitive data structures to select an appropriate structure for storing and managing the data and prepare a technical presentation justifying your choice based on the efficiency of data structure.
2	Choose a suitable linear or non-linear data structure for a navigation system that stores city connections and finds shortest routes between locations. Prepare a technical presentation explaining why it is appropriate.
3	A web browser supports navigating backward through previously visited pages. Recommend a suitable ADT and prepare a technical presentation explaining how it improves performance efficiency.
4	A digital library allows searching books using ISBN numbers quickly among 50,000 records. Select the most suitable data structure and Prepare a technical presentation justifying using time complexity comparison.

Code of Conduct:

I **Dhavanesh S S / 1972260011** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***Dhavanesh S S***

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Topic 1

1: Student Management System

Abstract

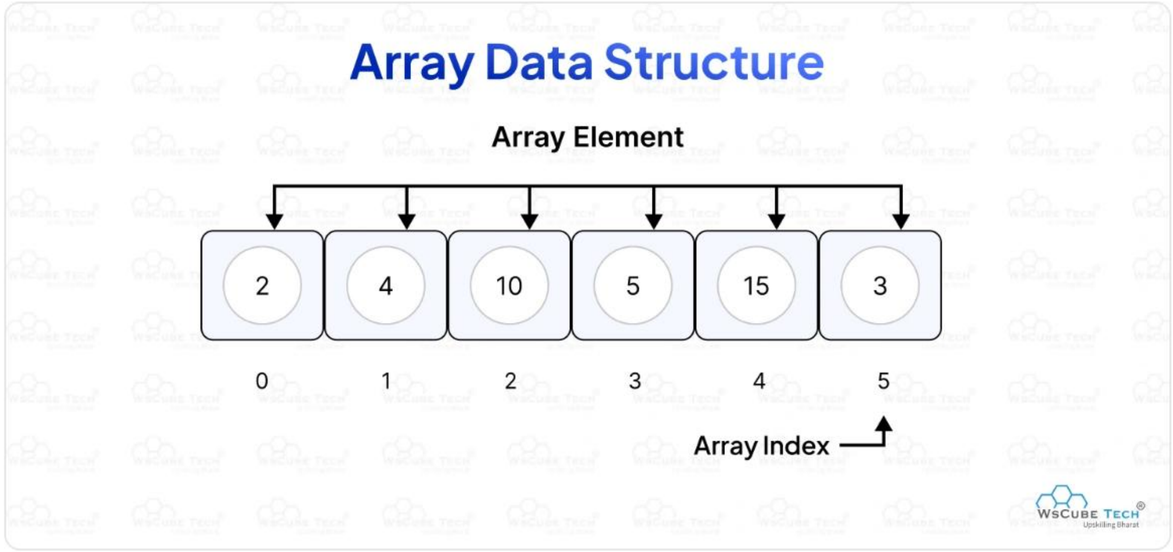
A Student Management System stores and manages the marks of 1,000 students. Selecting suitable primitive and non-primitive data structures helps organize student information, retrieve marks efficiently, and simplify record updates.

Problem Statement

Storing student marks without a proper data structure can make data access, searching, and updating inefficient. An appropriate structure is required to manage a large number of student records systematically.

Objectives

- Store student roll numbers, names, and marks.
- Apply primitive and non-primitive data structures.
- Improve record access and updating efficiency.
- Analyze operations using time complexity.



1. Student Management System Architecture

Existing System: Individual variables or unorganized records make managing many students difficult.

Proposed System: Use an array of student records, where each record contains a roll number, name, and marks.

Data Structures Used

Primitive: int, float, char or strings
Non-primitive: Array of student records.

Time Complexity

- Access by index: **O(1)**
- Display all records: **O(n)**
- Linear search by roll number: **O(n)**
- Update a known record: **O(1)**

Applications: Colleges, schools, examination systems.

Conclusion: An array of student records provides organized storage and fast indexed access when the number of students is known.

Array of Student Records (1,000 Students)

Index	Roll No	Name	Marks
0	101	Rahul Sharma	85
1	102	Priya Singh	92
2	103	Arjun Verma	78
...	...	...	...
999	1100	Sanjay J	90

Topic 2

2: City Navigation System

Abstract

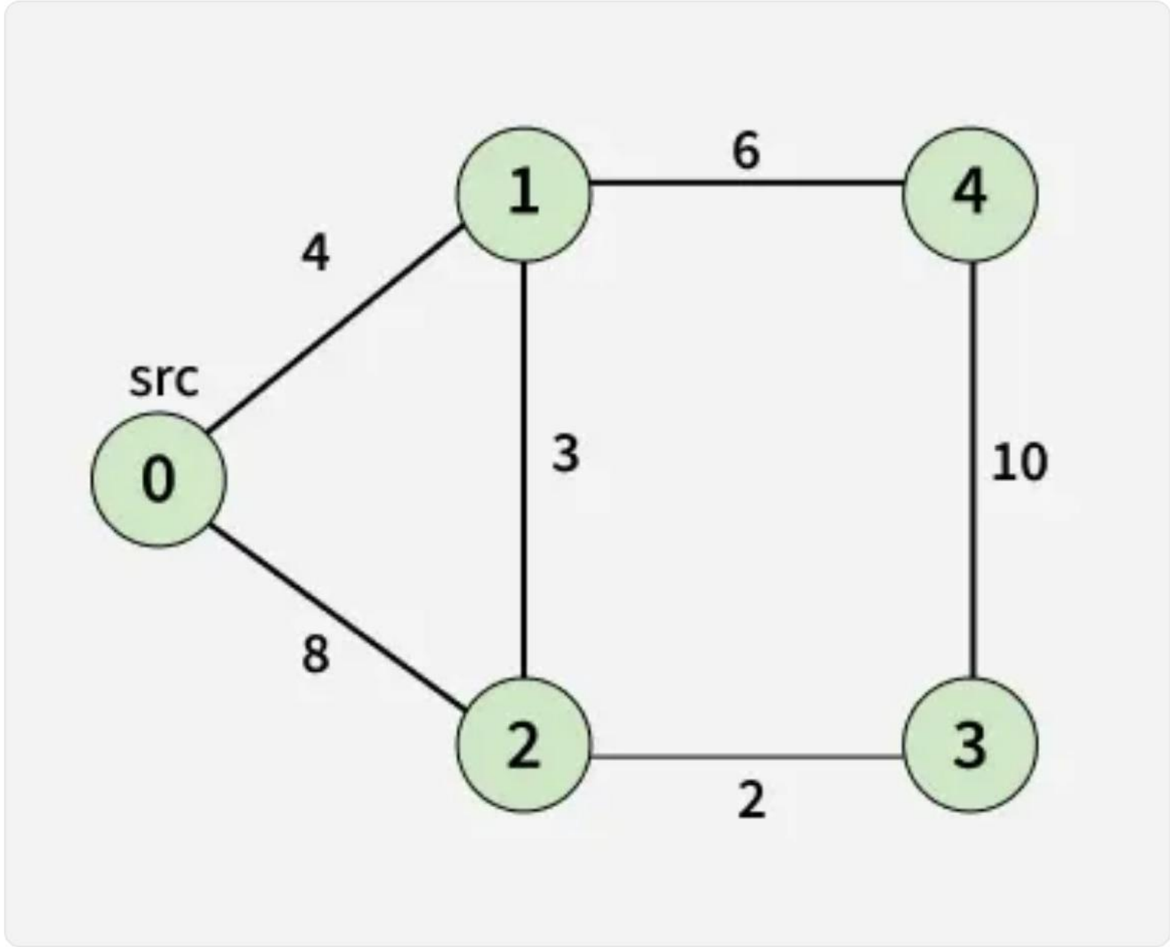
A City Navigation System represents cities and roads using a weighted graph. It calculates the shortest route between locations using a suitable graph algorithm, helping users travel efficiently.

Problem Statement

A city contains multiple interconnected roads and alternative routes. A suitable data structure is needed to represent these connections and identify the shortest route based on distance or travel cost.

Objectives

- To represent cities as vertices and roads as edges.
- To store road distances using edge weights.
- To find shortest routes efficiently.
- To reduce unnecessary travel distance.



2. Weighted Graph & Dijkstra Algorithm

Existing System: Simple lists or tables cannot naturally represent complex road connections and route alternatives.

Proposed System: Use a weighted graph with an adjacency list and Dijkstra's algorithm for non-negative road weights.

Methodology: Create graph → Enter source and destination → Apply Dijkstra's algorithm → Display shortest route and distance.

Time Complexity Analysis

Adjacency list storage: **O(V + E)**
Dijkstra with binary heap: **O((V + E) log V)**

Applications: GPS navigation, transport planning, delivery routing.

Conclusion: A weighted graph represents interconnected cities effectively and supports shortest-path calculation.

Weighted Graph Architecture

- **Vertices:** Cities
- **Edges:** Roads
- **Weights:** Distance / Travel Time
- **Representation:** Adjacency List

Shortest Route Example: A → C → E → D (12 km)

Topic 3

3: Web Browser Backward Navigation

Abstract

A web browser allows users to return to previously visited pages. A Stack Abstract Data Type (ADT) supports this feature through the Last In, First Out (LIFO) principle, allowing the most recently visited page to be retrieved first.

Problem Statement

Browsers must maintain page history and retrieve the previous page quickly. A suitable ADT is needed to manage this history without searching through all previously visited pages.

Objectives

- To store visited page history.
- To implement stack operations.
- To support efficient backward navigation.
- To analyze the time complexity of browser history operations.



3. Stack ADT Operations (LIFO)

Existing System: A simple unsorted history list may require searching or shifting elements to remove the latest page.

Proposed System: Use a Stack ADT implemented using an array or linked list.

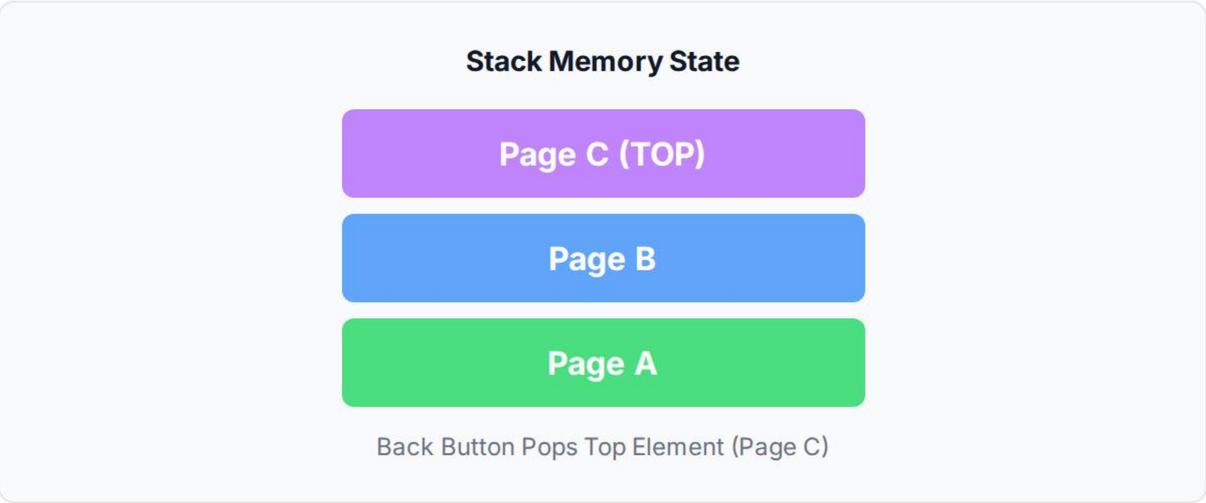
Methodology: Visit page → Push page into history → Click Back → Pop the most recent page → Display previous page.

Stack Operations Time Complexity

- Push: **O(1)**
- Pop: **O(1)**
- Peek: **O(1)**

Applications: Web browsers, undo operations, function-call management.

Conclusion: A stack efficiently supports backward navigation because its LIFO behavior matches the browser's history requirement.



Topic 4

4: Digital Library Search System

Abstract

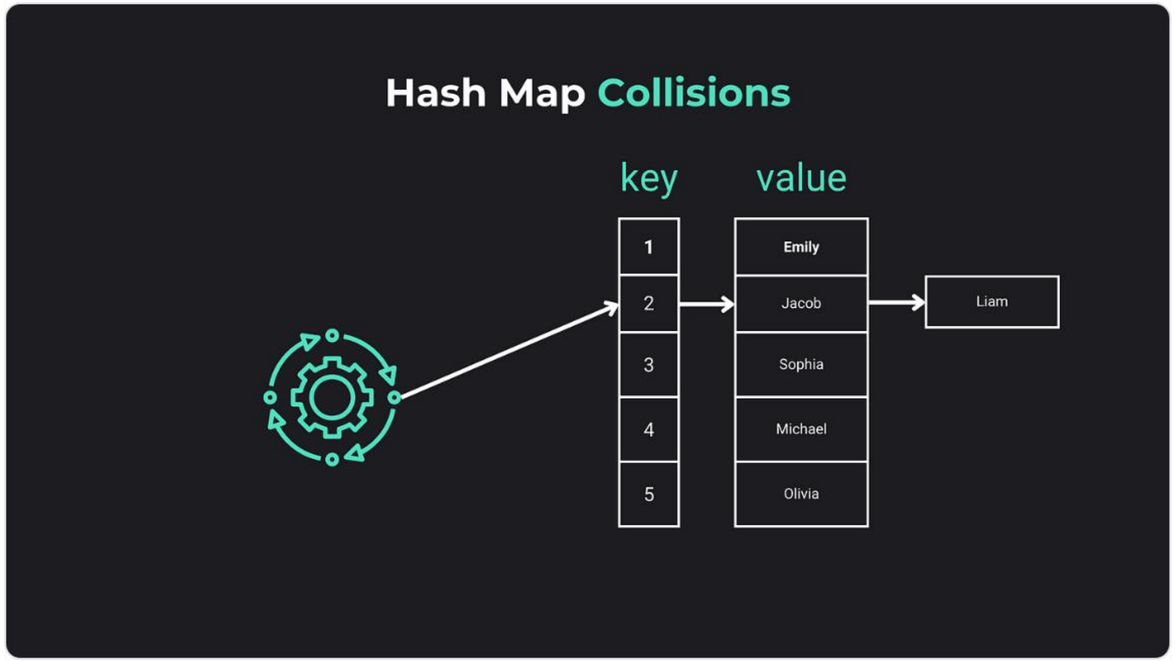
A Digital Library Search System manages 50,000 book records and enables quick retrieval using ISBN numbers. A hash table maps each ISBN to a storage location, supporting fast average-case lookup.

Problem Statement

Searching a large collection of books using linear search may require checking many records. An efficient data structure is required to locate books quickly using unique ISBN numbers.

Objectives

- Store ISBN numbers and book details.
- Implement ISBN-based lookup.
- Handle hash collisions efficiently.
- To compare search time complexity with alternative structures.



4. Hash Table & Complexity Comparison

Existing System: Linear search checks records sequentially and can take O(n) time.

Proposed System: Use a Hash Table, with ISBN as the key and book details as the value.

Time Complexity Comparison

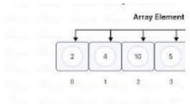
Data Structure	Search Complexity
Linear search	O(n)
Binary search on sorted array	O(log n)
Balanced BST	O(log n)
Hash table (average)	**O(1)**
Hash table (worst case)	O(n)

Applications: Digital libraries, bookshops, inventory systems.

Conclusion: A hash table provides fast average-case ISBN lookup when the hash function and collision-handling method are effective.

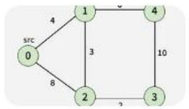


Image Sources



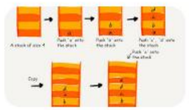
https://deen3evddmddt.cloudfront.net/uploads/content-images/array-data-structure.webp

Source: www.wscubetech.com



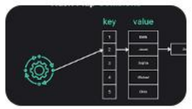
https://media.geeksforgeeks.org/wp-content/uploads/20251113155540209723/123456.webp

Source: www.geeksforgeeks.org



https://miro.medium.com/v2/resize:fit:1400/1*ej5pxeBGAuUGP7DSCilVOA.png

Source: blog.devgenius.io



https://miro.medium.com/1*iCOku96N9Cf7l4lPrxkANA.png

Source: medium.com